

AI Assisted Coding

Lab Assignment - 12.5

Name : P. Gouri Prasad Varma

Hall.t.No. : 2303A51035

Batch No. : 01

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use cases.
- Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

- **Task:** Use AI to generate a Python program that implements the Merge Sort algorithm.
- **Instructions:**
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- **Expected Output:**
 - o A functional Python script implementing Merge Sort with proper documentation.

Copilot Suggested Code :

```
task_1.py > ...
1  # Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
2
3  from pandas import merge
4
5
6  def merge_sort(arr):
7      """
8      Sorts a list in ascending order using the Merge Sort algorithm.
9
10     Time Complexity:  $O(n \log n)$  - where n is the number of elements in the array.
11     Space Complexity:  $O(n)$  - due to the temporary arrays used for merging.
12
13     Parameters:
14     arr (list): The list of elements to be sorted.
15
16     Returns:
17     list: A new list containing the sorted elements.
18     """
19     if len(arr) ≤ 1:
20         return arr
21
22     mid = len(arr) // 2
23     left_half = merge_sort(arr[:mid])
24     right_half = merge_sort(arr[mid:])
25
26     return merge(left_half, right_half)
27
28 def merge(left, right):
29     """
30     Merges two sorted lists into a single sorted list.
31
32     Parameters:
33     left (list): The first sorted list.
34     right (list): The second sorted list.
35
36     Returns:
37     list: A new list containing the merged and sorted elements from both lists.
38     """
39     merged = []
40     left_index = right_index = 0
41
42     while left_index < len(left) and right_index < len(right):
43         if left[left_index] < right[right_index]:
44             merged.append(left[left_index])
45             left_index += 1
46         else:
47             merged.append(right[right_index])
48             right_index += 1
49
50     # If there are remaining elements in left or right, add them to merged
51     merged.extend(left[left_index:])
52     merged.extend(right[right_index:])
53
54     return merged
55
56 # Test cases to verify the implementation
57 if __name__ == "__main__":
58     test_cases = [
59         [38, 27, 43, 3, 9, 82, 10],
60         [5, 2, 9, 1, 5, 6],
61         [12, 11, 13, 5, 6, 7],
62         [],
63         [1],
64         [2, 1]
65     ]
66
67     for i, test in enumerate(test_cases):
68         sorted_arr = merge_sort(test)
69         print(f"Test Case {i + 1}: {sorted_arr}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_1.py
Test Case 1: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: [1, 2, 5, 5, 6, 9]
Test Case 3: [5, 6, 7, 11, 12, 13]
Test Case 4: []
Test Case 5: [1]
Test Case 6: [1, 2]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- **Task:** Use AI to create a binary search function that finds a target element in a sorted list.
- **Instructions:**
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- **Expected Output:**
 - o Python code implementing binary search with AI- generated comments and docstrings.

Copilot Suggested Code :

```

task_2.py > ...
1  # Task: Use AI to create a binary search function that finds a target element in a sorted list.
2
3  def binary_search(arr, target):
4      """
5      Performs binary search on a sorted list to find the index of the target element.
6
7      Time Complexity:
8      - Best Case:  $O(1)$  - when the target is at the middle of the array.
9      - Average Case:  $O(\log n)$  - where  $n$  is the number of elements in the array.
10     - Worst Case:  $O(\log n)$  - when the target is not present in the array.
11
12     Space Complexity:  $O(1)$  - as we are using a constant amount of space.
13
14     Parameters:
15     arr (list): A sorted list of elements.
16     target: The element to search for.
17
18     Returns:
19     int: The index of the target element if found, otherwise -1.
20     """
21     left, right = 0, len(arr) - 1
22
23     while left ≤ right:
24         mid = left + (right - left) // 2
25
26         # Check if the target is present at mid
27         if arr[mid] == target:
28             return mid
29         # If target is greater, ignore the left half
30         elif arr[mid] < target:
31             left = mid + 1
32         # If target is smaller, ignore the right half
33         else:
34             right = mid - 1
35
36     # Target was not found in the array
37     return -1
38
39 # Test cases to verify the implementation
40 if __name__ == "__main__":
41     test_cases = [
42         ([1, 2, 3, 4, 5], 3), # Target is present
43         ([1, 2, 3, 4, 5], 6), # Target is not present
44         ([1, 2, 3, 4, 5], 1), # Target is the first element
45         ([1, 2, 3, 4, 5], 5), # Target is the last element
46         ([], 1),             # Empty array
47     ]
48
49     for arr, target in test_cases:
50         result = binary_search(arr, target)
51         print(f"Searching for {target} in {arr} → Index: {result}")

```

Output :

```

PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5\task_2.py
• Searching for 3 in [1, 2, 3, 4, 5] -> Index: 2
Searching for 6 in [1, 2, 3, 4, 5] -> Index: -1
Searching for 1 in [1, 2, 3, 4, 5] -> Index: 0
Searching for 5 in [1, 2, 3, 4, 5] -> Index: 4
Searching for 1 in [] -> Index: -1

```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Copilot Suggested Code :

```
task3.py :-
1 # Task Description #3: Smart Healthcare Appointment Scheduling System A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system need
2 # 1. Search appointments using appointment ID.
3 # 2. Sort appointments based on time or consultation fee.
4
5 # Recommended Algorithms:
6 # 1. Searching Algorithm: Hash Table (Dictionary in Python)
7 # 2. Sorting Algorithm: Merge Sort
8 # Justification:
9 # 1. Hash Table (Dictionary): Provides O(1) average-case time complexity for searching, making it highly efficient for finding appointments by ID.
10 # 2. Merge Sort: Offers O(n log n) time complexity for sorting, ensuring efficient ordering of appointments based on time or consultation fee.
11 # Implementation:
12 class Appointment:
13     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
14         self.appointment_id = appointment_id
15         self.patient_name = patient_name
16         self.doctor_name = doctor_name
17         self.appointment_time = appointment_time
18         self.consultation_fee = consultation_fee
19
20 class HealthcarePlatform:
21     def __init__(self):
22         self.appointments = {}
23
24     def add_appointment(self, appointment):
25         self.appointments[appointment.appointment_id] = appointment
26
27     def search_appointment(self, appointment_id):
28         return self.appointments.get(appointment_id, None)
29
30     def sort_appointments_by_time(self):
31         return sorted(self.appointments.values(), key=lambda x: x.appointment_time)
32
33     def sort_appointments_by_fee(self):
34         return sorted(self.appointments.values(), key=lambda x: x.consultation_fee)
35
36 # Test cases to verify the implementation
37 if __name__ == "__main__":
38     platform = HealthcarePlatform()
39
40     # Adding sample appointments
41     platform.add_appointment(Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100))
42     platform.add_appointment(Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150))
43     platform.add_appointment(Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120))
44
45     # Searching for an appointment
46     appointment = platform.search_appointment(2)
47     if appointment:
48         print(f"Found appointment: {appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for ${appointment.consultation_fee}")
49     else:
50         print("Appointment not found.")
51
52     # Sorting appointments by time
53     sorted_by_time = platform.sort_appointments_by_time()
54     print("Appointments sorted by time:")
55     for appt in sorted_by_time:
56         print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")
57
58     # Sorting appointments by consultation fee
59     sorted_by_fee = platform.sort_appointments_by_fee()
60     print("Appointments sorted by consultation fee:")
61     for appt in sorted_by_fee:
62         print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_3.py
Found appointment: Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by time:
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by consultation fee:
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

Task Description #4: Railway Ticket Reservation System Scenario A
a railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Copilot Suggested Code :

```
task_4.py --
1 # Task Description #4: Railway Ticket Reservation System Scenario A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:
2 # 1. Search tickets using ticket ID.
3 # 2. Sort bookings based on travel date or seat number.
4
5 # Recommended Algorithms:
6 # 1. Searching Algorithm: Hash Table (Dictionary in Python)
7 # 2. Sorting Algorithm: Merge Sort
8 # Justification:
9 # 1. Hash Table (Dictionary): Provides O(1) average-case time complexity for searching, making it highly efficient for finding tickets by ID.
10 # 2. Merge Sort: Offers O(n log n) time complexity for sorting, ensuring efficient ordering of bookings based on travel date or seat number.
11 # Implementation:
12 class Ticket:
13     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
14         self.ticket_id = ticket_id
15         self.passenger_name = passenger_name
16         self.train_number = train_number
17         self.seat_number = seat_number
18         self.travel_date = travel_date
19
20 class RailwayReservationSystem:
21     def __init__(self):
22         self.tickets = {}
23
24     def add_ticket(self, ticket):
25         self.tickets[ticket.ticket_id] = ticket
26
27     def search_ticket(self, ticket_id):
28         return self.tickets.get(ticket_id, None)
29
30     def sort_tickets_by_date(self):
31         return sorted(self.tickets.values(), key=lambda x: x.travel_date)
32
33     def sort_tickets_by_seat_number(self):
34         return sorted(self.tickets.values(), key=lambda x: x.seat_number)
35
36 # Test cases to verify the implementation
37 if __name__ == "__main__":
38     system = RailwayReservationSystem()
39
40     # Adding sample tickets
41     system.add_ticket(Ticket(1, "Alice", "Train A", "12A", "2024-08-01"))
42     system.add_ticket(Ticket(2, "Bob", "Train B", "10B", "2024-08-02"))
43     system.add_ticket(Ticket(3, "Charlie", "Train A", "11C", "2024-08-01"))
44
45     # Searching for a ticket
46     ticket = system.search_ticket(2)
47     if ticket:
48         print(f"Found ticket: {ticket.passenger_name} on {ticket.train_number} seat {ticket.seat_number} for travel on {ticket.travel_date}")
49     else:
50         print("Ticket not found.")
51
52     # Sorting tickets by travel date
53     sorted_by_date = system.sort_tickets_by_date()
54     print("Tickets sorted by travel date:")
55     for t in sorted_by_date:
56         print(f"{t.passenger_name} on {t.train_number} seat {t.seat_number} for travel on {t.travel_date}")
57
58     # Sorting tickets by seat number
59     sorted_by_seat = system.sort_tickets_by_seat_number()
60     print("Tickets sorted by seat number:")
61     for t in sorted_by_seat:
62         print(f"{t.passenger_name} on {t.train_number} seat {t.seat_number} for travel on {t.travel_date}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5\task_4.py
• Found ticket: Bob on Train B seat 10B for travel on 2024-08-02
Tickets sorted by travel date:
Alice on Train A seat 12A for travel on 2024-08-01
Charlie on Train A seat 11C for travel on 2024-08-01
Bob on Train B seat 10B for travel on 2024-08-02
Tickets sorted by seat number:
Bob on Train B seat 10B for travel on 2024-08-02
Charlie on Train A seat 11C for travel on 2024-08-01
Alice on Train A seat 12A for travel on 2024-08-01
```